Information Leakage in Backtesting

Weiguan Wang* Johannes Ruf†

June 24, 2022

Abstract

Testing the performance of statistical models with historical time series requires careful handling of the data. Even if a dataset is seemingly completely separated in an in-sample and an out-of-sample set information may be leaked. Such leakage can lead to a significant overestimation of the out-of-sample performance of a predictive model. We provide experimental evidence to illustrate how randomised data splits lead to overfitting in the presence of time series structure. The experiment is set up in the framework of option replication, with real-world and simulated data.

Keywords: Data snooping; Hedging; Information leakage; Overfitting; Pseudo real-time; Time series

1 Introduction

Backtesting is concerned with testing a predictive model on historical data. Such data usually are subject to a time series structure. Handling data incorrectly can introduce a strong bias in the evaluation of a model's performance. This is true even if the model of interest concerns only one period and hence is a priori not exposed to the time series' auto correlation. We illustrate this issue in the context of evaluating the performance of a hedging strategy that replicates the price of an option after one trading period.

It is well understood that the estimation and evaluation steps should not be performed on the same data. Hence, a study's dataset is usually split into two parts, namely the in-sample set and the out-of-sample set (Moran [1973], Stone [1974]). The former is used to estimate ('train') model parameters, while the latter is used to measure the model performance (i.e., the generalisation error). Information leakage is introduced if the split does not take into account the intrinsic time series structure, for example, if the dataset is randomly split. In this case, the in-sample set may contain some information from the out-of-sample set that would not be available if the data split was done in pseudo real-time, which puts the earlier part of the data into the in-sample and the later part into the out-of-sample set.

We thank Johannes Muhle-Karbe and James Wolter for helpful discussions on the subject matter of this paper. The code to reproduce the results in this paper can be found at https://github.com/weiguanwang/Information_Leakage_in_Backtesting.git. A shorter version of this paper can be found on the journal "Quantitative Finance", under the title "A note on spurious model selection" (Wang and Ruf [2022]).

*School of Economics, Shanghai University. Email: weiguanwang@shu.edu.cn

†Department of Mathematics, London School of Economics and Political Science. Email: j.ruf@lse.ac.uk

The importance of preserving pseudo real-time might seem highly relevant after this primer. However, the issue is often obscured when studying complex models without a relevant time structure (e.g., studying models concerned with only a single time period). Indeed, this work is motivated by the existence of numerous published articles that split time-series data randomly. We believe this is usually done with the best intentions, sometimes with statistical cross validation in mind and sometimes by carelessly using standard software. For example, the Python package 'scikit-learn,' one of the standard tools in machine learning, provides the function 'train_test_split,' which shuffles data by default before splitting them. In addition, for most machine learning applications outside of finance, e.g., for image recognition, a shuffle-and-split approach is standard.

Information leakage due to a faulty split into in-sample and out-of-sample sets is often disguised. As we shall see below it can be quite significant, nevertheless. It leads to an overestimated model performance on the out-of-sample set, as its samples are not anymore independent from the in-sample set. Thus the estimate of the generalisation error is biased downwards. Arguably more importantly, such information leakage can have more impact on a complex model than on a small parametric model. Indeed, a complex model has more parameters that can adjust to additional information. As the experiments below illustrate, this then can lead to wrong conclusions. For example, artificial neural networks (ANNs) might seem to perform better than linear models on the out-of-sample set. However, deployed later in production and in real time these more complex models perform worse as they now cannot rely anymore on the leaked information of the out-of-sample set that was only available in the backtesting step.

Our example to illustrate these issues concerns the hedging of European options, which are financial products that give the owner the right to buy or sell a certain stock at a pre-determined price. Given the price of a European option written on a stock, the goal is to determine a trading strategy (holding a certain number of units of the stock and the risk-free asset) that in some sense optimally replicates the option value on the next day. The available trading strategies are modelled as a class of function of observables. The aim is to choose this function and to evaluate its performance. We consider two classes of such functions: (1) the class of linear functions of some given variables describing the characteristics of the option; (2) a class of ANNs that map these variables in a nonlinear way to a hedging strategy. We demonstrate the information leakage for randomised data splits both under simulated and real-world data.

Related literature

Usually in the presence of a single time series (in contrast to panel data as in the experiments below), several papers in the literature discuss and argue in favour of block cross-validations. While an improper cross-validation might lead to overfitted models (Opsomer et al. [2001], Bergmeir and Benítez [2012]), blocked forms of cross-validation are argued to perform favourably in out-of-sample testing that preserves pseudo real-time (Burman et al. [1994], Racine [2000], Hall et al. [2004], Bergmeir et al. [2014], Bergmeir et al. [2018]).

This paper also relates to the idea of 'dataset shift.' This concept describes the situation when "the data available for model-building [...] are not strictly representative of the data on which the model will ultimately be deployed," according to Moreno-Torres et al. [2012]; see also Quiñonero-Candela et al. [2009] and Gama et al. [2014]. A dataset shift deteriorates a model's performance out-of-sample. This issue can arise for various reasons; for example by a non-stationary environment. The information leakage demonstrated here occurs when a dataset

shift is removed by a faulty in-sample/out-of-sample data split, which would be observed when backtesting in pseudo-real time. This leads to an over-optimistic model performance during backtesting.

By highlighting the importance of preserving the time structure in historical data, this study complements the important research in financial econometrics that highlights the pitfalls of backtesting and data pre-processing. Let us provide a small selection of those references that are most relevant for econometric and financial applications. A large part of this literature concerns 'specification search' or 'data snooping,' i.e., the invalidity of statistical statements due to their formulation only after looking at the observational data of interest and the formulation of several statistical hypotheses. Leamer [1978] is one of the first to provide a systematic analysis using a Bayesian framework. Lo and MacKinlay [1990] show that sorting on empirical characteristics of stock returns can yield misleading conclusions when backtesting portfolio strategies. White [2000] provides a bootstrap algorithm to correct for data snooping. Dimson and Marsh [1990] discuss, among others, the look-ahead bias when backtesting prediction models. A look-ahead bias is introduced when a time series structure is broken, similar to the question studied in this paper. As an example, they discuss how using the overall mean of a time series (i.e, across the in-sample and out-of-sample sets) improves a model's ability to forecast the quantity of interest. The issue of survivorship bias has also been widely recognised, for example in Brown et al. [1992] in the context of examining the performance of mutual funds. The cited works are instances of many excellent papers that discuss the repercussions of various forms of data snooping, in the context of backtesting trading strategies.

Arnott et al. [2019] formulate a point that is close in spirit to this paper: In the presence of two time series (i.e., corresponding to two different countries) training a model on one and testing it on the other one might not suffice to avoid overfitting. Cohen et al. [2021] provide a valuable list of potential data pre-processing errors. The present article adds to this literature on backtesting by highlighting by how much results can be biased when the time structure of data is broken. The danger here arises from a researcher's wrong belief that they did a proper out-of-sample test.

Outline of paper

The rest of the paper is organised as follows. Section 2 formalises the hedging problem, formulates two statistical models to estimate optimal hedging ratios, introduces the datasets, and describes the possible data splits and an additional simulated feature in more detail. Section 3 introduces the experimental design to show the emergence of information leakage through random data splits. Section 4 presents the experimental results for the problem of option replication. Section 5 concludes.

2 Backtesting hedging of options: description and data

This section first introduces the hedging problem in more detail and the statistical models considered. Then it describes the data, how they can be split into an in-sample and an out-of-sample set, and discusses the 'tagging' of the samples with an independently sampled process.

2.1 The hedging task

We consider the task of hedging a European call or put option over a one-day period. Such options are financial instruments that give the owner the right to buy (for calls) or sell (for puts) a unit of the 'underlying' in the future with a pre-determined price. The underlying below will denote a unit of the S&P 500 index, one of the most commonly watched financial indices. The market price of such an option changes over its lifetime. An institution that owns options is exposed to such market risk. To reduce this risk, the institution can actively trade the underlying ('hedge'). The institution has to determine how much of the underlying it wants to trade. Here the hedging strategy δ is assumed chosen from a large class of functions by backtesting. Each of these functions takes as input a set of variables that describe the characteristics of the option and the underlying.[1]

Using the framework from Ruf and Wang [2021], we consider the hedging error

$$V_1^\delta = \delta S_1 + (1 + r\Delta t)\,(C_0 - \delta S_0) - C_1. \tag{1}$$

This corresponds to the end-of-period wealth of an institution that bought δ shares of the underlying at price S_0 and is short a call, sold at the price C_0. Over the period, the difference $C_0 - \delta S_0$ is invested in a risk-free asset paying an annualised rate r. We consider $\Delta t = 1/253$, equivalent to a one-day hedging period. The goal is to minimise the mean-squared hedging error (MSHE) by choosing an appropriate δ. To reduce the non-stationarity of Dollar prices we normalise the hedging error; i.e., we shall consider $100 \times V_1^\delta / S_0$ instead of V_1^δ throughout. This normalisation 'removes the units' and allows a comparison over time; see also Section 2.3 in Ruf and Wang [2021].

A classical choice of the hedging strategy is the Black-Scholes (BS) Delta, given by

$$\delta_{\mathrm{BS}} = \mathbf{N}\left(\frac{\log(S_0/K) + (R + \sigma_{\mathrm{impl}}^2/2)\tau}{\sigma_{\mathrm{impl}}\sqrt{\tau}}\right), \tag{2}$$

where $\mathbf{N}$ denotes the cumulative standard normal distribution function, τ the time-to-maturity in year fraction, σ_{impl} the annualised implied volatility of the option, K its strike price, and R the risk-free interest rate corresponding to the option's maturity. When data are generated from the BS model and frictionless continuous trading is allowed, the BS Delta is the optimal hedging strategy in the sense that it minimizes the MSHE (the MSHE becomes zero).

When hedging is only allowed once in a time period and/or data are not generated from the BS model, the literature proposes a variety of function classes ('models') for the hedging strategy δ to improve the BS hedge. They range from linear or quadratic functions of the data (e.g., Hull and White [2017] or Ruf and Wang [2021]) to more complex ANNs (see Ruf and Wang [2020]). The goal then is to determine the hedging strategy δ (within a given function class) that minimises the MSHE. This can be achieved either by ordinary least squares in the

[1] An option is characeterized by its type (call / put), exercise style (European / American), strike price (pre-determined price to trade the underlying), and maturity (the date when the option expires). To assist the risk management of options, traders use the first- and second-order sensitivities of option prices with respect to the underlying's price or the option's implied volatility. These sensitivities relevant in this study are Delta, Vega, and Vanna. Here Delta (resp., Vega) denotes the sensitivity of the option price with respect to the underlying's price (resp., the implied volatility), and Vanna denotes the sensitivity of Delta with respect to the implied volatility. For example, if the implied volatility increases by some factor x then the option price is expected to increase roughly by Vanna times x. These sensitivities are here always computed under the Black-Scholes model.

linear case or by stochastic gradient descent in the case of ANNs. This step is done on an in-sample set. An out-of-sample set is then used to obtain an estimate of the MSHE achieved by the optimised hedging strategy. This estimate allows to compare the performance of different models (i.e., a linear hedging strategy or a hedging strategy specified by an ANN).

2.2 Statistical models for hedging

Below we compare two statistical models (function classes for the hedging strategy δ). These two models are among the best performing models studied in Ruf and Wang [2021]. The first model (LR, for 'linear regression') assumes that the hedging strategy δ is a linear function of the option's Delta, Vega, and Vanna. Those three numbers only depend on the information available at the beginning of each period and describe the sensitivity of the option price with respect to changes in the underlying's price and the implied volatility. The second model is a fully-connected feed-forward ANN with two hidden layers, each with 30 nodes[2]. Inputs to this ANN are Delta, Vega, and square root of total implied variance $\sigma_{\text{impl}}\sqrt{\tau}$. This number describes the annualized standard deviation of the underlying, multiplied by the square root of the time until the option matures. In summary, the two model classes being studied are described by

$$\delta_{\text{LR}} = f_{\text{LR}}(\delta_{\text{BS}}, \mathcal{V}_{\text{BS}}, \text{Vanna}_{\text{BS}}); \qquad \delta_{\text{ANN}} = f_{\text{ANN}}(\delta_{\text{BS}}, \mathcal{V}_{\text{BS}}, \sigma_{\text{impl}}\sqrt{\tau}).$$

Here, the notations $\mathcal{V}_{\text{BS}}$ and Vanna_{BS} represent the BS Vega and Vanna respectively, calculated under the BS model with the option's option implied volatility.

Note the the linear model only uses three parameters, namely the coefficients in front of the three sensitivities. In contrast, the ANN-based model uses $(3+1)\times 30+(30+1)\times 30+(30+1) = 1081$ parameters. We omit further details on those two models as they are not relevant for the discussions below. The interested reader can find those details in the provided GitHub code that accompanies this paper or in Ruf and Wang [2021].

2.3 Simulated and real-world data

The experiment below is repeated on simulated and on real-world data. The simulated data are generated from the BS model.[3] The real-world data are end-of-day S&P 500 option data (SPX) obtained from OptionMetrics (2010-2019). This data is split in 14 overlapping windows (each rolled forward by 6 months) to allow several replications of the experiment below. Each of these datasets constitute panel data, i.e., cross sections of time series. There exist, at any point of time, several options which are different in strike and/or maturity.

Let us next describe a specific data point. Each point describes one out-of-the-money option[4] over one period (1 day). The data point contains the option price at the beginning and end of the period and the underlying's price at the end of the period (at its beginning, the price is always normalised to 100). Moreover, the data point includes a flag indicating whether the

[2]The learning rate (during the stochastic gradient descent) and the L^2 regularisation parameter are both set to 10^{-4}.

[3]The annualised rate of return is set to 10% and the annualised volatility to 20%. Options are generated for various strikes according to the CBOE rules; see http://www.cboe.com/products/stock-index-options-spx-rut-msci-ftse/s-p-500-index-options/s-p-500-options-with-a-m-settlement-spx/spx-options-specs.

[4]One says a put is out-of-the-money if the strike price is less than the price of the underlying at the moment. Similarly, a call is out-of-the-money if the strike price is greater than the price of the underlying.

option is a call or a put, the risk-free rate, the strike and time-to-maturity of the option, its implied volatility, and its BS sensitivities. Further details on these datasets are provided in Ruf and Wang [2021].

2.4 Separation into in-sample and out-of-sample sets

It is common practice to split data into in-sample and out-of-sample sets when measuring a model's performance or comparing different models, in particular, when the models are of different complexity. For panel data as used here, different kinds of splits can be employed, e.g., chronological or random, as shown in Figure 1.

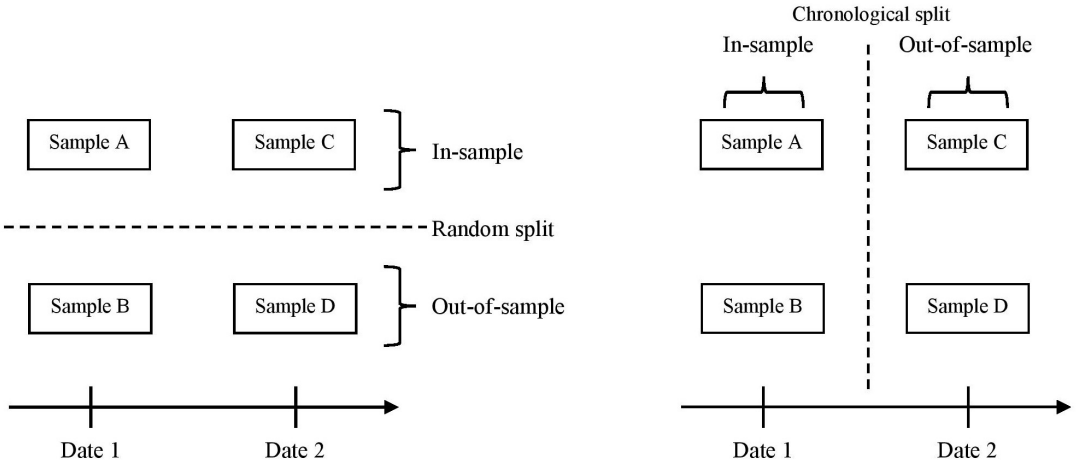

Figure 1: Illustration of the random (left) and chronological (right) data splits. Each day has two one-period samples. Samples A and B (as well as samples C and D) are from two different options traded on the same day. Samples A and C (as well as samples B and D) correspond to the same option traded in two different days. A chronological split implies that all samples belonging to an early day constitute the in-sample set and those belonging to a late day constitute the out-of-sample set. A random split implies that samples belonging to the same day could (but not always) appear in both in-sample and out-of-sample sets.

When performing a chronological split ('pseudo real-time'), first a critical date is determined. Samples from days before this point constitute the in-sample set and the remaining ones the out-of-sample set. Alternatively, the data could be split at random into in-sample and out-of-sample sets. In this approach, the in-sample and out-of-sample sets are also disjoint. However, we shall argue that such an approach introduces significant information leakage. Indeed, on each day several options are traded. Hence samples from the same day might show up in both the in-sample and out-of-sample sets simultaneously.

Both data splits frequently appear in the literature, including in very prestigious journals (see the survey Ruf and Wang [2020] for many examples of papers that use one or the other split). This is unfortunate as the experiments below show that a random split causes significant information leakage. This in turn leads to wrong conclusions of the performances of the models under consideration.

The splits in the experiments are implemented as follows. First, the data are split chronologically so that in each dataset the first 83% (=5/6) of days are assigned to the in-sample set. The remaining days are allocated to the out-of-sample set. We shall also perform a random split to illustrate the associated information leakage. This is done as follows. For each of the simulated and 14 real-world datasets the samples are randomly reassigned to the in-sample and out-of-sample sets. This random permutation is done under the constraint that the sizes of the two sets do not change.[5]

2.5 Tagging the data

We shall see that information leakage caused by the wrong data split becomes even more significant if the data are additionally 'tagged'. To explain what we mean consider the situation that at any trading day we have an additional observation, say the daily value of the VIX (Volatility Index), which indicates the market risk-neutral expectation of the future S&P 500 volatility. In practice, the VIX is derived from the implied volatilities of S&P 500 index call and put options, and differs from individual option implied volatility or the S&P 500 historical volatility.

Note that all samples from the same trading day will be equipped with the same VIX value. Let us now assume that we use this observation as an additional feature. Concerning the two models from Section 2.2, this implies that now the linear model has four and the ANN model $(4+1)\times 30+(30+1)\times 30+(30+1)=1111$ parameters.

If the data are randomly split, the same day, and hence the same VIX value might appear both in the in-sample and out-of-sample sets. The experiment below will show that including such a feature inflates the out-of-sample performance of the models, purely due to information leakage.

In the experiment we shall use independently sampled random variables as an additional feature. We call this feature 'fake VIX' to remind ourselves that it has nothing to do with any real-world observations. More precisely, the 'fake VIX' observations are the daily samples of an Ornstein-Uhlenbeck process[6], which is simulated independently from any of the other data. Clearly, adding this 'fake VIX' value as a feature should not help at all in reducing the MSHE, as the corresponding Ornstein-Uhlenbeck process has nothing to do with the datasets.

3 Experimental design

The goal of this paper is to demonstrate the information leakage caused by a faulty in-sample/out-of-sample data split. To this end, we run an experiment for each dataset, as explained in the following. Each dataset will be pre-processed in four different ways ('configurations'). The first two configurations involve a chronological split. The remaining two rely on a random split; i.e., the samples are randomly assigned to the in-sample and out-of-sample set, as explained in Subsection 2.4. The second and the fourth configurations differ from the other two in so far as their samples are equipped with the 'fake VIX' value introduced in Subsection 2.5.

To summarise, the four configurations are as follows:

[5]The training of the ANN requires a further split of the in-sample set to obtain a validation set (see Section 5.3 in Goodfellow et al. [2016]). We use 20% of the in-sample set for validation. In the random permutation, 20% of the the newly generated in-sample set is randomly assigned to the validation set.

[6]We use 1 for the rate of mean reversion, 25 for the volatility coefficient, 13 for the starting value, and 15 for the long-term mean of this process. Other parameter values lead to similar results.

1. The '*Baseline*' configuration corresponds to the standard setup. The dataset is separated chronologically into an in-sample and an out-of-sample set.

2. The '*VIX*' configuration takes the 'Baseline' configuration, but adds the simulated 'fake VIX' variable as an additional feature in the linear regression and the ANN.

3. The '*Permute*' configuration corresponds to the random split into in-sample and out-of-sample sets.

4. The '*Permute + VIX*' configuration is as the '*Permute*' configuration, but now with the 'fake VIX' variable as an additional feature.

For the BS data, we repeat each configuration twenty times as follows. We first simulate a path P_0. Continuing from the last day of P_0, twenty more paths are simulated, denoted by P_i for $i \in \{1, ..., 20\}$. For the 'Baseline' and 'VIX' configurations, we fit only *once* the linear regression and the ANN on the in-sample set constructed from P_0. We then check these models on each of the out-of-sample sets constructed on the P_i's. For the 'Permute' and 'Permute + VIX' configurations, we take each pair (P_0, P_i) to construct a pair of the randomly split in-sample and out-of-sample sets. Then the models are fit and tested on *every* such pair.

For each of the 14 real-world datasets we run each of the configurations once. (Additional checks with different seedings for the random permutations lead to the same conclusions.)

4 Experimental proof of the presence of information leakage

This section reports and interprets the out-of-sample performance of the linear model and the ANN for each dataset and each of the four configurations of the previous section. The performance is measured in terms of the out-of-sample MSHE. More precisely, we report the improvement in out-of-sample MSHE relative to the BS Delta, i.e.,

$$\frac{\text{MSHE}(\delta_*) - \text{MSHE}(\delta_{\text{BS}})}{\text{MSHE}(\delta_{\text{BS}})},$$

where $*$ represents the linear or the ANN model.

Figures 2 and 3 summarise the results on the BS and the S&P 500 datasets, respectively. The left panels display the relative improvement in MSHE relative to the BS Delta, averaged over the 20 simulated and 14 real-world datasets, respectively. The right panels in Figures 2 and 3 show these results broken down by permutation set (BS data) or time window (S&P 500 data). The time windows for the S&P 500 data are chronologically ordered; the permutation sets for the BS data are ordered by performance of the ANN in the 'Baseline' configuration. Each of the presented numbers in the right panels correspond to the additional improvement relative to the BS Delta due to the permutation and 'fake VIX' feature, e.g. for the 'Permute + VIX' configuration:

$$\frac{\text{MSHE}(\delta_*^{\text{Permute+VIX}}) - \text{MSHE}(\delta_{\text{BS}}^{\text{Permute+VIX}})}{\text{MSHE}(\delta_{\text{BS}}^{\text{Permute+VIX}})} - \frac{\text{MSHE}(\delta_*^{\text{Baseline}}) - \text{MSHE}(\delta_{\text{BS}}^{\text{Baseline}})}{\text{MSHE}(\delta_{\text{BS}}^{\text{Baseline}})}.$$

A value of -20% for 'Permute + VIX' means that this configuration adds an extra 20% to the relative improvement in the 'Baseline' configuration. This difference is due to information leakage, induced by the permutation and 'tagging' of the samples.

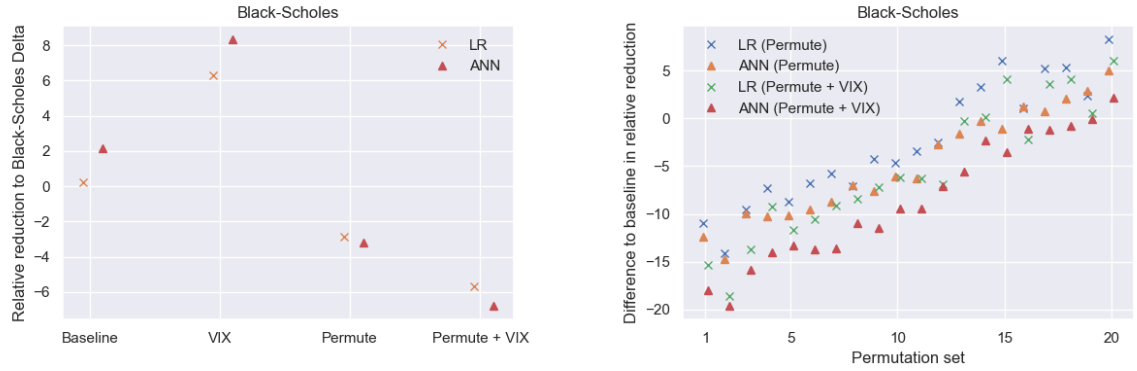

Figure 2: Illustration of information leakage when failing to take into account the time series structure of a simulated dataset. The left panel displays the relative reduction in MSHE over using the BS Delta for each of the four configurations described in the previous section. In the 'Baseline' configuration, neither the ANN nor the linear regression improve the MSHE relative to the BS Delta. Adding the 'fake VIX' feature reduces furthermore their performance since this feature is simulated independently of the data, and thus, pure noise. However, when the in-sample and out-of-sample sets are randomly permuted, both the ANN and the linear regression outperform the BS Delta. Moreover, now the 'fake VIX' feature reduces the error further, illustrating the information leakage induced by the random permutations.
The right panel displays the relative reduction in MSHE by permuting in-sample and out-of-sample sets. The relative reduction is improved more in the case of the ANN than in the case of the linear regression, and the 'fake VIX' helps both the linear regression and the ANN. The permutation sets are ordered from left to right by the relative reduction of the ANN in the 'Baseline' configuration. (Recall that the 'Baseline' configuration uses the same ANN but has a different out-of-sample error for each permutation set.) In other words, the permutation set built from the pair (P_0, P_i) on which the ANN performs the worst relative to BS in the 'Baseline' configuration is on the left.

Let us now summarise our observations in three points.

1) In the 'VIX' configuration, both the linear regression and the ANN perform worse than the BS Delta benchmark in the 'Baseline' configuration. This effect is stronger for the ANN than for the linear regression, at least on the S&P 500 dataset. An explanation is easy. The additional feature is simulated completely independently from the data. Hence, it has no predictive power for the hedging ratio at all. Its inclusion adds additional noise, and the lower performance is due to an overfit of the training procedure, being more severe for the more complex ANN than for the linear regression with three/four parameters.

2) Even without using the 'fake VIX' as feature, the permuted datasets lead to a better performance relative to the BS Delta benchmark. This holds true in the case that the samples are generated by a time-homogeneous BS model. Since the discrete time steps are small, we know, a priori, that BS hedging is close to optimal. Nevertheless, instead of underperforming by about 0.2% for the linear regression and 2% for the ANN, the linear regression and ANN reduce the BS Delta benchmark in the BS data by about 3% after data permutation, with a larger relative improvement for the ANN. For the S&P 500 dataset, permuting samples allows the linear regression and ANN to reduce the BS Delta benchmark by about 23%, instead of about 19% in the 'Baseline' configuration.

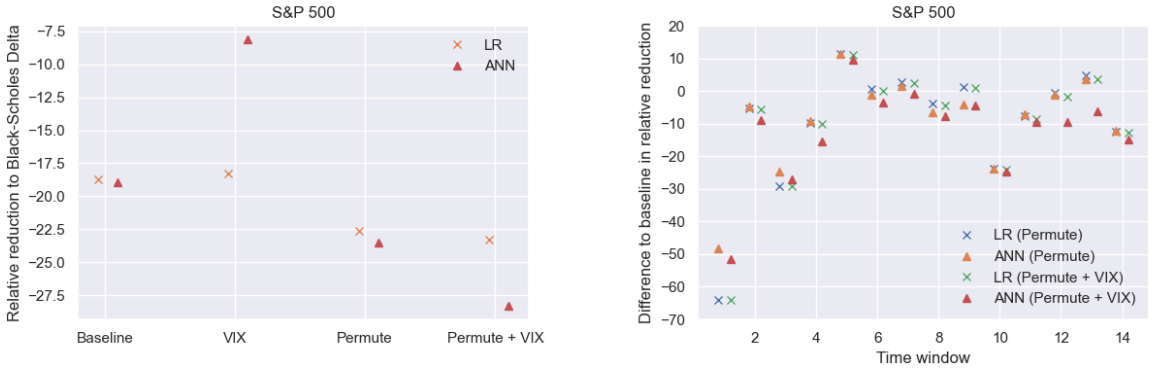

Figure 3: Illustration of information leakage when failing to take into account the time series structure of the S&P 500 dataset. See the caption of Figure 2 for explanations.

3) At first glance, the results with the 'fake VIX' included as an additional feature when the samples are permuted are startling. Both statistical models improve, but most dramatically the ANN, which now outperforms the BS Delta benchmark by about 7% in the BS simulated data and by about 29% in the S&P 500 data. What is going on? By construction, different samples have the same 'fake VIX' value. Indeed, each day has several options (corresponding to different strikes) but only one 'fake VIX' value. The random permutation now allows samples from the same day to appear both in the in-sample and out-of-sample sets. The presence of the additional feature makes it possible for the ANN (and partially also for the linear regression model) to understand from which day a sample is. In other words, the 'fake VIX' tags the different days and the models are able to pick up on it. This is relevant since on any specific day the underlying's price goes up or down (or, in case of the S&P 500 data, there is a certain shift in the implied volatility surface). This leaked information improves the models' hedging performance in backtesting but would of course not be available in real time.

5 Conclusion

This paper illustrates the relevance of preserving time structure between in-sample and out-of-sample sets. Even for a linear regression model with few parameters, a faulty data split may lead to remarkably overconfident estimates of the model's performance. In addition, a more complex model (such as an ANN) may capture the induced information leakage better. Thus one might wrongly conclude that such a model outperforms the simpler one in a direct comparison.

The information leakage is further reinforced when data are 'tagged' by the presence of an additional feature that takes the same value for different samples from the same date. Then random permutations make this feature informative (despite it having nothing to do with the data-generating mechanism). This yields a further seemingly important improvement for a model's performance, not achievable when applying the model in real time.

We have argued that an out-of-time validation technique (splitting data chronologically into in-sample and out-of-sample sets) is preferable to a random split. Other validation techniques might be possible and appropriate in different contexts; for example, in credit risk (see Sobehart et al. [2000] or Stein [2007]) or in order-book modelling (e.g., Sirignano and Cont [2018]), namely when it is of interest how a model performs not only in real time but also on new

subjects/assets.

References

R. Arnott, C. R. Harvey, and H. Markowitz. A backtesting protocol in the era of machine learning. *The Journal of Financial Data Science*, 1(1):64–74, 2019.

C. Bergmeir and J. M. Benítez. On the use of cross-validation for time series predictor evaluation. *Information Sciences*, 191:192–213, 2012.

C. Bergmeir, M. Costantini, and J. M. Benítez. On the usefulness of cross-validation for directional forecast evaluation. *Computational Statistics & Data Analysis*, 76:132–143, 2014.

C. Bergmeir, R. J. Hyndman, and B. Koo. A note on the validity of cross-validation for evaluating autoregressive time series prediction. *Computational Statistics & Data Analysis*, 120: 70–83, 2018.

S. J. Brown, W. Goetzmann, R. G. Ibbotson, and S. A. Ross. Survivorship bias in performance studies. *The Review of Financial Studies*, 5(4):553–580, 1992.

P. Burman, E. Chow, and D. Nolan. A cross-validatory method for dependent data. *Biometrika*, 81(2):351–358, 1994.

S. N. Cohen, D. Snow, and L. Szpruch. Black-box model risk in finance. arXiv:2102.04757, 2021.

E. Dimson and P. Marsh. Volatility forecasting without data-snooping. *Journal of Banking & Finance*, 14(2-3):399–421, 1990.

J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia. A survey on concept drift adaptation. *ACM Computing Surveys (CSUR)*, 46(4):1–37, 2014.

I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, 2016.

P. Hall, J. Racine, and Q. Li. Cross-validation and the estimation of conditional probability densities. *Journal of the American Statistical Association*, 99(468):1015–1026, 2004.

J. Hull and A. White. Optimal delta hedging for options. *Journal of Banking & Finance*, 82: 180–190, 2017.

E. E. Leamer. *Specification Searches: Ad Hoc Inference with Nonexperimental Data*. New York: John Wiley, 1978.

A. W. Lo and A. C. MacKinlay. Data-snooping biases in tests of financial asset pricing models. *The Review of Financial Studies*, 3(3):431–467, 1990.

P. A. Moran. Dividing a sample into two parts a statistical dilemma. *Sankhyā: The Indian Journal of Statistics, Series A*, pages 329–333, 1973.

J. G. Moreno-Torres, T. Raeder, R. Alaiz-Rodríguez, N. V. Chawla, and F. Herrera. A unifying view on dataset shift in classification. *Pattern Recognition*, 45(1):521–530, 2012.

J. Opsomer, Y. Wang, and Y. Yang. Nonparametric regression with correlated errors. *Statistical Science*, pages 134–153, 2001.

J. Quiñonero-Candela, M. Sugiyama, N. D. Lawrence, and A. Schwaighofer. *Dataset Shift in Machine Learning*. MIT Press, 2009.

J. Racine. Consistent cross-validatory model-selection for dependent data: hv-block cross-validation. *Journal of Econometrics*, 99(1):39–61, 2000.

J. Ruf and W. Wang. Neural networks for option pricing and hedging: a literature review. *Journal of Computational Finance*, 24(1):1–46, 2020.

J. Ruf and W. Wang. Hedging with linear regressions and neural networks. *Journal of Business & Economic Statistics*, SSRN 3580132, 2021. Forthcoming.

J. Sirignano and R. Cont. Universal features of price formation in financial markets: perspectives from deep learning. *Quantitative Finance*, 19:1449–1459, 2018.

J. R. Sobehart, S. C. Keenan, and R. Stein. Benchmarking quantitative default risk models: a validation methodology. *Moody's Investors Service*, 2000.

R. M. Stein. Benchmarking default prediction models: Pitfalls and remedies in model validation. *Journal of Risk Model Validation*, 1:77–113, 2007.

M. Stone. Cross-validatory choice and assessment of statistical predictions. *Journal of the Royal Statistical Society: Series B (Methodological)*, 36(2):111–133, 1974.

W. Wang and J. Ruf. A note on spurious model selection. *Quantitative Finance*, 2022. Forthcoming.

H. White. A reality check for data snooping. *Econometrica*, 68(5):1097–1126, 2000.